```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_re
```

```
heart_dataset = pd.read_csv('/content/heart_disease_data.csv')
```

```
# 1 = having disease,  0 = not having disease
heart_dataset.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca
0	63	1	3	145	233	1	0	150	0	2.3	0	0
1	37	1	2	130	250	0	1	187	0	3.5	0	0
2	41	0	1	130	204	0	0	172	0	1.4	2	0
3	56	1	1	120	236	0	1	178	0	0.8	2	0
4	57	0	0	120	354	0	1	163	1	0.6	2	0

Next steps: Generate code with `heart_dataset` New interactive sheet

```
heart_dataset.shape
```

```
(303, 14)
```

```
heart_dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trestbps  303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
 8   exang     303 non-null    int64
```

```
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64
 11  ca        303 non-null    int64
 12  thal      303 non-null    int64
 13  target    303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

```
heart_dataset.isnull().sum()
```

	0
age	0
sex	0
cp	0
trestbps	0
chol	0
fbs	0
restecg	0
thalach	0
exang	0
oldpeak	0
slope	0
ca	0
thal	0
target	0

dtype: int64

```
heart_dataset.describe()
```

	age	sex	cp	trestbps	chol	fbs	r
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.
mean	54.366337	0.683168	0.966997	131.623762	246.264026	0.148515	0.
std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.
25%	47.500000	0.000000	0.000000	120.000000	211.000000	0.000000	0.
50%	55.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.
75%	61.000000	1.000000	2.000000	140.000000	274.500000	0.000000	1.
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.

```
heart_dataset['target'].value_counts()
```

target	count
1	165
0	138

dtype: int64

- Defective Heart = 1
- Healthy Heart = 0

```
X = heart_dataset.drop(columns='target', axis=1)
Y = heart_dataset['target']
```

```
print(X)
print(Y)
```

```
     age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  \
0     63    1   3       145   233    1        0      150      0      2.3
1     37    1   2       130   250    0        1      187      0      3.5
2     41    0   1       130   204    0        0      172      0      1.4
3     56    1   1       120   236    0        1      178      0      0.8
4     57    0   0       120   354    0        1      163      1      0.6
..   ...  ...  ..       ...   ...  ...      ...      ...    ...      ...
298   57    0   0       140   241    0        1      123      1      0.2
299   45    1   3       110   264    0        1      132      0      1.2
300   68    1   0       144   193    1        1      141      0      3.4
301   57    1   0       130   131    0        1      115      1      1.2
```

```
302   57    0   1       130   236    0        0      174      0      0.0

     slope  ca  thal
0        0   0     1
1        0   0     2
2        2   0     2
3        2   0     2
4        2   0     2
..     ...  ..   ...
298      1   0     3
299      1   0     3
300      1   2     3
301      1   1     3
302      1   1     2

[303 rows x 13 columns]
0      1
1      1
2      1
3      1
4      1
      ..
298    0
299    0
300    0
301    0
302    0
Name: target, Length: 303, dtype: int64
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, strati
```

```
print(X.shape, X_train.shape, X_test.shape)
```

```
(303, 13) (242, 13) (61, 13)
```

```
model = LogisticRegression()
model.fit(X_train, Y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: C
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regressic
  n_iter_i = _check_optimize_result(
```

```
▾ LogisticRegression
LogisticRegression()
```

```
X_train_prediction = model.predict(X_train)
training_data_accuracy = accuracy_score(X_train_prediction, Y_train)
```

```
print('Accuracy on Training data : ', training_data_accuracy)
```

```
Accuracy on Training data :  0.8512396694214877
```

```
X_test_prediction = model.predict(X_test)
test_data_accuracy = accuracy_score(X_test_prediction, Y_test)
```

```
print('Accuracy on Test data : ', test_data_accuracy)
```

```
Accuracy on Test data :  0.819672131147541
```

Model is overfitting

```
cm = confusion_matrix(X_test_prediction, Y_test)
print('classification_report:' , cm)
```

```
classification_report: [[23  6]
 [ 5 27]]
```

```
classification_report = classification_report(X_test_prediction, Y_test)
print('classification_report:' , classification_report)
```

```
classification_report:               precision    recall  f1-score   support

           0       0.82      0.79      0.81        29
           1       0.82      0.84      0.83        32

    accuracy                           0.82        61
   macro avg       0.82      0.82      0.82        61
weighted avg       0.82      0.82      0.82        61
```

```
input_data = (62,0,0,140,268,0,0,160,0,3.6,0,2,2)

input_data_as_numpy_array = np.asarray(input_data)

input_data_reshaped = input_data_as_numpy_array.reshape(1,-1)

prediction = model.predict(input_data_reshaped)
print(prediction)

if (prediction[0]== 0):
  print('The Person does not have a Heart Disease')
else:
```

```
    print('The Person has Heart Disease')
```

```
[0]
The Person does not have a Heart Disease
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWa
  warnings.warn(
```

```
labels = ['Training Accuracy', 'Test Accuracy']
accuracy_scores = [training_data_accuracy, test_data_accuracy]

plt.figure(figsize=(8, 6))
plt.bar(labels, accuracy_scores, color=['skyblue', 'lightcoral'])
plt.ylim(0, 1) # Accuracy ranges from 0 to 1
plt.ylabel('Accuracy')
plt.title('Comparison of Training and Test Accuracy')
plt.show()
```

Start coding or generate with AI.